

# FLACtastic PlayerEngine

## A Technical Guide to Gapless Audio Playback

Understanding how the engine decodes, schedules, and plays audio tracks seamlessly.

### 1. Overview

PlayerEngine is a gapless audio playback system built on AVAudioEngine (Apple's real-time audio processing framework). Its main job is to decode FLAC, WAV, MP3, and other audio files, then feed decoded audio samples to the AVAudioEngine in real-time so the user hears continuous playback with no gaps or stuttering between tracks.

#### Key Design Goals:

- Gapless playback: seamless transitions between tracks
- Responsive seeking: fast scrubbing through tracks
- Queue management: support adding/removing tracks on-the-fly
- @MainActor integration: safe, synchronous SwiftUI integration
- Off-main-thread heavy work: decoding and file I/O don't block UI

### 2. Architecture Overview

PlayerEngine has two main processing threads:

#### Main Actor (UI Thread):

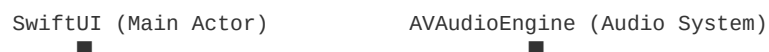
- Holds all mutable state (queue, currentIndex, playback status)
- Receives API calls from SwiftUI (play(), pause(), seek())
- Manages the tick task (50ms polling for playback position)
- Updates UI-facing properties (currentTrack, currentTime, etc.)

#### Decode Task (Background Thread):

- Long-lived Task running off-main-thread
- Continuously reads files from disk
- Converts audio format if needed (e.g., 48kHz → 44.1kHz)
- Schedules decoded buffers on AVAudioEngine
- Checks main-actor state live (queue, generation counter)

### 3. Playback Data Flow

When you call play(), here's what happens:





#### 4. State Management: Scheduled Entries

The engine tracks which audio has been scheduled and which is playing using a **ScheduledEntry**:

**struct ScheduledEntry:**

```
let track: Track // Which song
let startFrame: AVAudioFramePosition // First sample position in engine
var endFrame: AVAudioFramePosition // Last sample position
```

**Example:**

If Song A has 100,000 samples and Song B has 80,000 samples, and you play them gaplessly:

Song A: ScheduledEntry(track: A, startFrame: 0, endFrame: 100,000)  
Song B: ScheduledEntry(track: B, startFrame: 100,000, endFrame: 180,000)

Timeline:

The diagram shows a horizontal timeline with two segments. The first segment, labeled 'Song A Playing', starts at 0 and ends at 100,000. The second segment, labeled 'Song B Playing', starts at 100,000 and ends at 180,000. The timeline is represented by a series of small black squares along a horizontal axis.

The engine stores these entries and uses them to: (1) Detect when playback moves to the next track (2) Update `currentTime` and `currentTrack` (3) Clean up old entries to save memory

## 5. The Decode Loop: How Files Become Audio

The decode task runs this core loop:

```
while !Task.isCancelled {
  1. Ask main actor: "What's the next track to decode?"
    (Live read of _queue[trackIndex])

  2. Open file: AVAudioFile(track.url)

  3. Create entry: Add to scheduledEntries with endFrame = .max
    (.max is a sentinel meaning "still decoding")

  4. If file format differs, create AVAudioConverter
    (e.g., 48kHz mono → 44.1kHz stereo)

  5. Loop: Read chunks from file, convert, schedule
    • Read 8192 source samples
    • Convert to canonical format
    • Ask engine to play (schedule the buffer)
    • Wait for backpressure (throttle) if needed

  6. Finalize: Replace endFrame from .max to actual end

  7. Move to next track
}
```

### Two Decode Paths (Optimization):

The engine is smart about format conversion. If a file already matches the engine's canonical format (sample rate, channels, bit depth), it skips the converter entirely and reads directly to save CPU.

```
Path 1: Passthrough (fast, no conversion)
File (44.1kHz stereo) → Read directly → Schedule

Path 2: Converter (with format adaption)
File (48kHz mono) → Read → Convert to 44.1kHz stereo → Schedule
```

## 6. Generation Counter: Cancelling Stale Work

When you seek or change the queue, we need to throw away any partially-decoded audio. We do this with a **generation counter**:

```
Initial: generation = 0

User seeks forward:
■■ generation = 1 (bumped)
■■ Old decode task checks: "Is my generation (0) current (1)?"
■■ Answer: No! So it exits early
■■ New decode task spawned with generation = 1

User seeks again:
■■ generation = 2
■■ Both old tasks see mismatched generation and exit
```

This avoids the race condition where an old, slow decode tries to schedule audio after the user has already moved to a different part of the song.

## 7. Buffer Throttle: Backpressure Management

The decode task can run much faster than real-time. Without throttling, it would queue up 100MB of audio samples instantly, wasting memory and breaking seek responsiveness. The **BufferThrottle** holds it back.

- Limit: ~10 seconds of audio at the device's sample rate

When decode task schedules a buffer:

■ Check: Are we over the limit?

■ If yes: Pause the decode task (await) until playback

■ consumes some audio

■ If no: Continue

Example at 44.1kHz:

■ Limit =  $44,100 \text{ samples/sec} \times 10 \text{ sec} = 441,000 \text{ frames}$

■ Decode pauses if 441,001+ frames are scheduled but unplayed

## 8. Time Tracking: The Tick Task

While audio plays, the engine needs to constantly update the UI about the current playback position. This happens 20 times per second via the **tick task**.

### The Tick Loop (50ms intervals):

```
while !Task.isCancelled {  
    1. Ask AVAudioEngine: "Where are we in playback?"  
       (Read sampleTime from the player node)  
  
    2. Search scheduled entries:  
       "Which track is playing at sampleTime?"  
  
    3. If found:  
       ■■ Update currentTime (calculate from frame offset)  
       ■■ If track changed, update currentTrack and duration  
       ■■ Clean up old entries (ones before current position)  
  
    4. If at end and no new audio is decoding:  
       ■■ Set isPlaying = false  
       ■■ Stop the tick (no need to poll anymore)  
  
    5. Sleep 50ms, then repeat  
}
```

### Track Transitions (Automatic):

When playback crosses from Song A into Song B, the tick task notices the sampleTime has moved into Song B's ScheduledEntry. It automatically updates currentTrack and currentIndex. No explicit "next track" call needed.

## 9. Public API Methods

| Method                     | Purpose                                                                     |
|----------------------------|-----------------------------------------------------------------------------|
| setQueue(tracks, startAt:) | Replace entire queue, start at index. Flush old audio, begin decoding.      |
| play()                     | Start playback. Tell engine to begin consuming buffers.                     |
| pause()                    | Pause playback. Stop the tick task.                                         |
| seek(to:)                  | Jump to time. Flush, seek file, resume decoding from that position.         |
| appendTracks()             | Add to end without interrupting. Decode task picks up on next loop.         |
| insertTracks(at:)          | Insert mid-queue. Flush and re-decode from current position (audible seam). |
| next()                     | Skip to next track. Shortcut for setQueue(startAt: currentIndex + 1).       |
| previous()                 | Go to previous track or restart current if > 3 seconds in.                  |

## 10. Concurrency & Thread Safety

### The Challenge:

The main actor (UI thread) and decode task (background thread) access shared state. For example, the decode task reads `_queue` while the main thread might insert tracks. Swift 6 strict concurrency prevents race conditions.

### The Solution:

```
@MainActor class PlayerEngine
```

This annotation means:

- All properties are main-actor isolated
- All methods run synchronously on main actor
- Background code (decode task) uses "await MainActor.run" to access state
- Each access is serialized, no race conditions

Example:

```
// From decode task (background):
let nextTrack = await MainActor.run {
    self._queue[trackIndex] // Safe to read
}

// This await blocks the task until main actor is free,
// then atomically reads _queue
```

### Thread-Safe Primitives:

- `OSAllocatedUnfairLock`: Fast lock for generation counter (decode task checks current generation without blocking main thread)
- `UncheckedSendableBox`: Wrapper to pass non-Sendable values to tasks (AVAudioFile must be wrapped since it's not Sendable)
- Structured concurrency: `Task.detached`, `await`, `async/await` Enforces explicit handoffs between threads

## 11. Common Scenarios

### Scenario A: User plays a 3-track album

1. User clicks "Play" on Album with [Song A, Song B, Song C]
2. Main: `startFreshQueue([A, B, C], startAt: 0)`
  - `_queue = [A, B, C]`, `_currentIndex = 0`
3. Main: `startDecoding(from: 0)`
  - Decode task spawns
4. Decode: Open Song A, read 8192 samples, convert if needed
  - Schedule on engine
  - Loop until Song A is fully scheduled
5. Decode: Move to `trackIndex = 1` (Song B)
  - Read live from `_queue[1]`
  - Schedule Song B
6. Playback: Engine plays Song A

- ■ Tick task updates currentTime
- 7. Playback: Audio naturally transitions to Song B (no gap!)
  - ■ Tick task detects sampleTime crossed into Song B's entry
  - ■ Updates currentTrack = Song B
- 8. Decode: Finishes Song C
  - ■ Decode task sets isDecoding = false and exits
- 9. Playback: Reaches end of Song C
  - ■ Tick notices sampleTime past Song C's endFrame
  - ■ Sets isPlaying = false, stops tick

## Scenario B: User seeks mid-song

1. Song is playing at 30 seconds
2. User drags seek bar to 1:30
  - ■ Main: seek(to: 90)
3. Main: Bump generation (0 → 1)
4. Decode task: Notices generation mismatch
  - ■ Exits early, skipping finalization
5. Main: Flush engine (discard scheduled audio)
6. Main: startDecoding(from: currentIndex, seekOffset: 90)
  - ■ New decode task spawned with generation = 1
7. Decode: Open current track
  - ■ Seek file to frame position for 90 seconds
  - ■ Begin decoding from that position
8. Playback: Engine plays audio starting at the new position
  - ■ Tick task updates currentTime = 90 seconds

## Scenario C: User adds song to queue while playing

1. Currently playing Song A out of [A, B, C]
2. User right-clicks Song X, selects "Add to Queue"
  - ■ Main: addToQueue([X])
3. Main: \_queue.append(X) → [A, B, C, X]
4. Decode task (mid-way through Song C):
  - ■ Next loop iteration reads \_queue[2] = C
  - ■ If not yet reached X, continues decoding C
5. After Song C finishes:
  - ■ Next iteration reads \_queue[3] = X
  - ■ Begins decoding X
6. Audio: Song A → B → C → X (all gapless!)

## 12. Common Issues & Debugging

### Issue: Audio stuttering

**Cause:** Decode task is too slow (file I/O blocked, converter overloaded). Or throttle limit too low, causing frequent pauses. **Fix:** Profile decode speed, check sample rate mismatches, increase throttle limit.

### Issue: Gap between tracks

**Cause:** Decode finishes too late, or scheduledEntries cleanup is premature. **Fix:** Ensure decode task is started early (before queue fills). Check that endFrame finalization completes before entry is cleaned up.

### Issue: currentTime stuck or jumping

**Cause:** Stale ScheduledEntry with endFrame = .max (decoder crashed). Or sampleTime from engine is unreliable. **Fix:** Verify decoder always sets endFrame. Check that graph.currentPlayerSampleTime() is valid.

### Issue: Seek doesn't work

**Cause:** Old decode task still running (generation not checked). Or file not opened correctly with file.framePosition. **Fix:** Ensure generation mismatch causes early return. Verify file.framePosition is set before reading.

## 13. Summary: The Big Picture

PlayerEngine orchestrates real-time audio playback by keeping three things in sync:

1. FILE DECODING (Decode Task)
  - Reads files, converts formats, feeds to AVAudioEngine
2. AUDIO PLAYBACK (AVAudioEngine + Audio Hardware)
  - Consumes buffered samples, sends to speakers
3. TIME TRACKING (Tick Task)
  - Polls playback position, updates UI, detects track changes

These run independently and async, coordinated via:

- ScheduledEntry: Maps audio samples to track boundaries
- Generation counter: Cancels stale work
- Throttle: Prevents buffering explosion
- @MainActor: Safe state access from background threads

The result: gapless playback that feels instantaneous, responsive to user input (seeks, queue changes), and efficient (no excessive buffering or CPU waste).

*For more details, read the inline code comments and explore the AVAudioEngine docs.*